

Modul Kuliah – IF231303

Arsitektur Perangkat Lunak

Alfa Yohannis

**Program Studi Teknologi Informasi
Universitas Pradita**

Daftar Isi

1	Pengenalan Arsitektur Perangkat Lunak dan Client-Server	5
1.1	Pendahuluan	5
1.2	Aspek yang Dinilai pada Sebuah Arsitektur	6
1.3	Studi Kasus Singkat	6
1.4	Arsitektur Client-Server	7
1.5	Dua Biaya yang Melekat	8
1.6	Kelebihan dan Kekurangan	9
1.7	Ringkasan	9
1.8	Latihan	10
2	Layered Architecture	13
2.1	Pendahuluan	13
2.2	Pembagian Lapisan	14
2.3	Aturan Ketergantungan	15
2.4	Lapisan Tertutup dan Lapisan Terbuka	15
2.5	Kelebihan dan Kekurangan	17
2.6	Ringkasan	18
2.7	Latihan	18
3	Model-View-*	23
3.1	Pendahuluan	23
3.2	Peran Ketiga Komponen	24
3.3	Empat Varian dan Arah Ketergantungannya	24
3.4	Ketergantungan View sebagai Pembeda	25
3.5	Kelebihan dan Kekurangan	26
3.6	Ringkasan	26
3.7	Latihan	27
4	Ports and Adapters	31
4.1	Pendahuluan	31
4.2	Port dan Adapter	32

4.3	Menerima Adapter sebagai Argumen	33
4.4	Tiga Akibat yang Dapat Diukur	33
4.5	Kelebihan dan Kekurangan	34
4.6	Ringkasan	35
4.7	Latihan	35
Daftar Pustaka		39

Bab 1

Pengenalan Arsitektur Perangkat Lunak dan Client-Server

Tujuan Pembelajaran

1. Menjelaskan arti arsitektur perangkat lunak beserta aspek yang membedakannya dari rancangan rinci.
2. Membedakan tanggung jawab klien dari tanggung jawab server pada arsitektur client-server.
3. Mengukur waktu bolak-balik satu permintaan dan perubahannya ketika jumlah klien bertambah.

1.1 Pendahuluan

Setiap sistem perangkat lunak punya arsitektur, baik dirancang maupun tidak. Arsitektur adalah susunan bagian-bagian besar sistem beserta aturan hubungan di antaranya. Susunan itu menentukan hal-hal yang mahal diubah di kemudian hari, misalnya batas antar modul dan cara modul berkomunikasi.

Yang membedakan arsitektur dari rancangan rinci adalah biaya perubahannya. Mengganti nama sebuah fungsi murah. Memindahkan aturan bisnis dari satu bagian ke bagian lain juga masih murah. Sebaliknya, memecah satu sistem menjadi beberapa layanan terpisah menyentuh hampir seluruh berkas. Keputusan yang mahal diubah itulah yang disebut keputusan arsitektural.

Bab ini membuka dengan pengertian arsitektur perangkat lunak beserta aspek yang dinilainya, lalu menutup dengan bentuk pertama yang dibahas dalam modul ini, yaitu client-server. Bentuk tersebut dipilih lebih dahulu karena hampir seluruh

bentuk lain pada bab-bab berikutnya tetap memakainya sebagai dasar. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [1].

1.2 Aspek yang Dinilai pada Sebuah Arsitektur

Sebuah arsitektur tidak dinilai benar atau salah, melainkan sesuai atau tidak sesuai dengan kebutuhan. Penilaiannya memakai sejumlah aspek, dan aspek-aspek tersebut sering saling bertentangan. Tabel 1.1 merangkum lima aspek yang paling sering dipakai.

Tabel 1.1: Lima aspek yang paling sering dipakai untuk menilai arsitektur

Aspek	Pertanyaannya	Diukur dengan
Kemudahan perawatan	Berapa banyak berkas berubah untuk satu permintaan perubahan?	Jumlah berkas dan baris
Kemampuan skala	Apa yang terjadi ketika jumlah pengguna naik sepuluh kali?	Latensi pada beban bertingkat
Kinerja	Berapa lama satu permintaan dilayani?	Median dan persentil ke-95
Ketersediaan	Apa yang gagal ketika satu bagian mati?	Jumlah proses yang terhenti
Kemudahan pengujian	Berapa besar bagian yang dapat diuji tanpa infrastruktur?	Cakupan pengujian

Aspek-aspek tersebut jarang dapat dipenuhi seluruhnya sekaligus. Menaikkan kemampuan skala biasanya menurunkan kemudahan perawatan, sebab sistem harus dipecah menjadi lebih banyak bagian. Menaikkan kinerja sering menurunkan kemudahan pengujian, sebab jalan pintas mulai bermunculan. Pekerjaan seorang arsitek karena itu bukan mencari susunan terbaik, melainkan memilih aspek mana yang dikorbankan.

Setiap bab pada modul ini mengukur dua atau tiga aspek dari tabel tersebut, lalu melaporkan angkanya. Cara itu dipakai agar perbandingan antar bentuk arsitektur berdiri di atas angka, bukan di atas pendapat.

1.3 Studi Kasus Singkat

Tiga kejadian berikut menunjukkan bahwa keputusan arsitektural berakibat nyata, bukan sekadar soal kerapian kode.

Netflix. Pada 2008 layanan Netflix berhenti cukup lama akibat kerusakan pada basis data tunggalnya. Seluruh sistem saat itu berupa satu kesatuan, dan kegagalan satu bagian menghentikan seluruhnya. Sesudah kejadian tersebut

sistem dipecah menjadi banyak layanan terpisah, sehingga kegagalan satu layanan tidak lagi menghentikan yang lain.

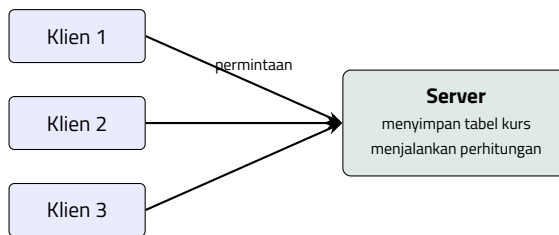
Healthcare.gov. Pada peluncurannya tahun 2013 situs pendaftaran asuransi kesehatan Amerika Serikat tersebut nyaris tidak dapat dipakai. Salah satu sebabnya, sistem memanggil banyak sistem luar secara berurutan dan menunggu seluruh jawabannya sebelum menampilkan halaman. Satu sistem luar yang lambat membuat seluruh halaman ikut lambat.

Sistem perbankan. Sistem yang memproses transaksi secara berkelompok pada akhir hari menunda pembaruan saldo sampai kelompok tersebut selesai diproses. Perpindahan ke pemrosesan berbasis event membuat saldo diperbarui segera setelah transaksi terjadi. Bab 9 membahas bentuk tersebut secara khusus.

Ketiganya memperlihatkan pola yang sama. Yang gagal bukan algoritmanya, melainkan susunan bagian-bagiannya beserta cara mereka saling menunggu.

1.4 Arsitektur Client-Server

Client-server membagi sistem menjadi dua peran. Klien meminta, server melayani. Gambar 1.1 menunjukkan susunannya. Banyak klien memanggil satu server, dan seluruh panah berawal dari klien.



Gambar 1.1: Banyak klien memanggil satu server. Seluruh permintaan berawal dari klien, dan server tidak pernah memanggil klien

Pembagian tersebut menentukan siapa menyimpan apa. Server menyimpan data bersama dan menjalankan aturan bisnis. Klien menyimpan keadaan tampilan saja. Karena tabel kurs hanya ada di server, memperbaruinya cukup dilakukan di satu tempat, dan seluruh klien langsung memakai nilai baru tersebut.

Analogi yang mendekati adalah warung dengan satu kasir. Pembeli antri dan memesan, kasir yang menghitung dan menyimpan uang. Pembeli tidak perlu tahu harga pokok setiap barang, dan perubahan harga cukup diberitahukan kepada kasir. Kelemahannya juga sama, yaitu ketika pembeli banyak, antrian memanjang walaupun setiap pesanan sebenarnya cepat dilayani.

Listing 1.1 memperlihatkan inti server pada contoh bab ini. Server membaca

parameter permintaan, menghitung, lalu membalas dalam bentuk JavaScript Object Notation (JSON).

```

1 def do_GET(self):
2     """Membaca parameter permintaan, menghitung, lalu membalas dalam JSON."""
3     parameter = parse_qs(urlparse(self.path).query)
4     asal = parameter.get("asal", ["USD"])[0]
5     tujuan = parameter.get("tujuan", ["IDR"])[0]
6     nominal = float(parameter.get("nominal", ["1"])[0])
7
8     hasil = hitung_konversi(asal, tujuan, nominal)
9     if hasil is None:
10         self.kirim(404, {"galat": "Kurs tidak tersedia"})
11     else:
12         self.kirim(200, {"asal": asal, "tujuan": tujuan, "hasil": hasil})

```

Listing 1.1: Penanganan satu permintaan pada server, tersedia di `sources/chapter01/server.py`

Klien pada contoh tersebut sangat tipis. Klien hanya menyusun alamat, mengirim permintaan, lalu menampilkan jawaban. Tidak ada tabel kurs dan tidak ada perhitungan di sisi klien.

1.5 Dua Biaya yang Melekat

Pemisahan klien dan server menimbulkan dua biaya yang tidak dapat dihindari. Keduanya menjadi bahan pengukuran pada latihan bab ini.

Biaya perjalanan. Setiap permintaan harus menempuh jaringan dua kali, yaitu pergi dan pulang. Pada server di komputer yang sama biaya ini kecil, tetapi tetap ada. Pada server yang berada di kota lain, biaya ini menjadi bagian terbesar dari waktu tanggap, dan tidak dapat diperkecil dengan mempercepat perhitungannya.

Biaya antrean. Satu server melayani banyak klien. Ketika permintaan datang lebih cepat daripada kemampuan server melayaninya, sisanya menunggu. Server pada contoh bab ini melayani satu permintaan pada satu waktu, sehingga gejala tersebut muncul jelas begitu jumlah klien bertambah.

Tabel 1.2 merangkum hasil pengukuran nyata pada contoh bab ini. Kolom terakhir memperlihatkan gejala antrean tersebut.

Tabel 1.2: Latensi terhadap jumlah klien serentak pada server contoh

Jumlah klien serentak	Median (ms)	Terbesar (ms)
1	0,240	8,027
2	0,363	0,848
4	0,777	1,748
8	1,472	1011,138
16	1,481	1019,625

Angka pada kolom terakhir melonjak tajam mulai delapan klien. Nilai median hanya naik dari 0,24 menjadi 1,48 milidetik, sedangkan nilai terbesarnya mencapai seribu milidetik. Sebagian permintaan karena itu menunggu sangat lama, sementara sebagian besar lainnya tetap cepat. Gejala tersebut tidak terlihat bila yang dilaporkan hanya nilai rata-rata, dan itulah alasan pengukuran pada modul ini selalu menyertakan nilai terbesar atau persentil ke-95.

1.6 Kelebihan dan Kekurangan

Tabel 1.3 merangkum untung rugi bentuk client-server.

Tabel 1.3: Kelebihan dan kekurangan arsitektur client-server

Kelebihan	Kekurangan
Data bersama disimpan di satu tempat, sehingga pembaruan cukup sekali	Server menjadi satu-satunya titik yang bila mati menghentikan seluruh layanan
Aturan bisnis tidak dapat diubah dari sisi klien	Setiap permintaan menanggung biaya perjalanan jaringan
Klien menjadi ringan dan mudah diganti	Server menjadi antrean bersama ketika klien bertambah
Perangkat klien tidak perlu bertenaga besar	Menambah kemampuan server lebih mahal daripada menambah klien

1.7 Ringkasan

Arsitektur adalah susunan bagian besar sistem beserta aturan hubungan di antaranya. Yang membedakannya dari rancangan rinci adalah biaya perubahan. Keputusan yang murah diubah bukan keputusan arsitektural, sedangkan keputusan yang menyentuh hampir seluruh berkas termasuk di dalamnya.

Sebuah arsitektur dinilai memakai beberapa aspek yang saling bertentangan, yaitu kemudahan perawatan, kemampuan skala, kinerja, ketersediaan, dan kemudahan pengujian. Aspek-aspek tersebut jarang dapat dipenuhi seluruhnya sekaligus, sehingga pekerjaan seorang arsitek adalah memilih aspek mana yang dikorbankan.

Ketiga studi kasus pada bab ini memperlihatkan pola yang sama. Yang gagal bukan algoritmanya, melainkan susunan bagian-bagiannya beserta cara mereka saling menunggu. Perbaikannya pun bukan mempercepat perhitungan, melainkan mengubah susunannya.

Client-server membagi sistem menjadi peminta dan pelayan. Server menyimpan data bersama dan menjalankan aturan bisnis, sedangkan klien menyimpan keadaan tampilan saja. Pembagian tersebut membuat pembaruan data cukup

dilakukan di satu tempat, dan membuat aturan bisnis tidak dapat diubah dari sisi klien.

Pembagian itu menimbulkan dua biaya tetap, yaitu perjalanan jaringan dan antrean di server. Pengukuran pada bab ini menunjukkan biaya antrean tidak terlihat pada nilai median, tetapi tampak jelas pada nilai terbesar. Karena itu setiap pengukuran pada modul ini melaporkan sebaran, bukan satu angka rata-rata.

1.8 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter01/`, tetapi menysasar tiga masalah yang berbeda. Latihan 1 memeriksa pemisahan tanggung jawab. Latihan 2 mengukur biaya perjalanan. Latihan 3 mengukur perilaku server ketika klien bertambah. Seluruh pengukuran hanya dijalankan terhadap server milik sendiri, tidak terhadap sistem milik pihak lain. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Server dijalankan lebih dahulu pada satu terminal seperti pada Listing 1.2, lalu skrip lainnya pada terminal kedua.

Listing 1.2: Menjalankan server pada terminal pertama

```
1 cd sources/chapter01
2 python server.py
```

Latihan 1: Memetakan Tanggung Jawab

Latihan ini membuktikan bahwa klien tidak menyimpan maupun menghitung apa pun. Langkah kerjanya sebagai berikut.

1. Jalankan `python klien.py USD IDR 100` pada terminal kedua, lalu catat jawabannya.
2. Buka `sources/chapter01/klien.py`, lalu cari tabel kurs di dalamnya.
3. Ubah satu nilai pada tabel kurs di `sources/chapter01/server.py`, jalankan ulang server, lalu jalankan klien tanpa mengubah apa pun.
4. Matikan server, lalu jalankan klien sekali lagi dan catat yang terjadi.

Tabel 1.4 memperlihatkan bentuk pengisian yang diharapkan.

Isikan hasil pengamatan pada Tabel 1.5.

Jawab dua pertanyaan berikut. Pertama, jelaskan keuntungan menyimpan tabel kurs hanya di server. Kedua, jelaskan kerugiannya berdasarkan yang terjadi ketika server dimatikan.

Tabel 1.4: Contoh hasil Latihan 1 yang sudah terisi

Pertanyaan	Jawaban	Bukti
Klien menyimpan tabel kurs	tidak	Tidak ada kamus KURS di <code>klien.py</code>
Klien ikut berubah setelah kurs diubah	tidak	Jawaban berubah tanpa menyunting klien

Tabel 1.5: Lembar isian Latihan 1

Pertanyaan	Jawaban	Bukti
Klien menyimpan tabel kurs	...	...
Klien menghitung konversi	...	...
Klien ikut berubah setelah kurs diubah	...	...
Yang terjadi ketika server dimatikan	...	...

Latihan 2: Mengukur Biaya Perjalanan

Latihan ini mengukur waktu bolak-balik satu permintaan. Langkah kerjanya sebagai berikut.

1. Jalankan `python ukur_latensi.py`, lalu catat kelima angka yang dilaporkan.
2. Ulangi pengukuran sebanyak tiga kali.
3. Bandingkan nilai terkecil dengan nilai terbesar pada setiap pengukuran.
4. Tambahkan perhitungan berulang di dalam `hitung_konversi` agar server bekerja lebih lama, lalu ukur kembali.

Tabel 1.6 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 1.6: Contoh hasil Latihan 2 yang sudah terisi

Pengukuran	Median	Persentil ke-95	Terbesar
Ke-1, 200 permintaan	0,237 ms	0,351 ms	7,391 ms

Isikan hasil pengukuran pada Tabel 1.7.

Tabel 1.7: Lembar isian Latihan 2

Pengukuran	Median	Persentil ke-95	Terbesar
Ke-1	...	...	...
Ke-2	...	...	...
Ke-3	...	...	...
Setelah server dibuat lebih lambat	...	...	...

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, nilai terbesar pada pengukuran pertama jauh melebihi mediannya, sehingga jelaskan sebabnya. Kedua, jelaskan bagian mana dari waktu tersebut yang tetap

ada walaupun perhitungan di server dipercepat.

Latihan 3: Menambah Jumlah Klien

Latihan ini membuktikan bahwa server adalah antrean bersama. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python ukur_beban.py`, lalu catat median dan nilai terbesar pada setiap jumlah klien.
- 2. Tandai jumlah klien saat nilai terbesar mulai melonjak.
- 3. Bandingkan pertumbuhan median terhadap pertumbuhan nilai terbesar.
- 4. Jelaskan mengapa keduanya tumbuh dengan pola yang berbeda.

Tabel 1.8 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 1.8: Contoh hasil Latihan 3 yang sudah terisi

Jumlah klien	Median (ms)	Terbesar (ms)
4	0,777	1,748
8	1,472	1011,138

Isikan hasil pengukuran pada Tabel 1.9.

Tabel 1.9: Lembar isian Latihan 3

Jumlah klien	Median (ms)	Terbesar (ms)
1	...	...
2	...	...
4	...	...
8	...	...
16	...	...
Jumlah klien saat lonjakan mulai	...	

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, median tumbuh perlahan sedangkan nilai terbesar melonjak, sehingga jelaskan apa yang dialami sebagian kecil permintaan tersebut. Kedua, jelaskan mengapa gejala itu hilang bila hanya nilai rata-rata yang dilaporkan. Ketiga, dengan menggabungkan hasil ketiga latihan, sebutkan aspek dari Tabel 1.1 yang paling tertekan pada arsitektur client-server beserta angka yang menjadi dasarnya.

Bab 2

Layered Architecture

Tujuan Pembelajaran

1. Menjelaskan pembagian tanggung jawab pada keempat lapisan *layered architecture* dan arah ketergantungan yang berlaku di antaranya.
2. Membedakan lapisan tertutup dari lapisan terbuka, lalu menentukan kapan masing-masing bentuk layak dipilih.
3. Mengukur jumlah fungsi penerus, pelanggaran arah ketergantungan, dan selisih waktu antara kedua bentuk pada satu aplikasi yang sama.

2.1 Pendahuluan

Layered architecture menata sebuah aplikasi menjadi beberapa lapisan yang bertumpuk. Setiap lapisan mengerjakan satu jenis urusan teknis, lalu memanggil lapisan di bawahnya bila membutuhkan sesuatu. Bentuk ini juga dikenal sebagai arsitektur *n-tier*. Bentuk ini menjadi pilihan pertama bagi sebagian besar sistem, dan sering dipakai tanpa pertimbangan panjang ketika tim belum memutuskan arsitektur lain.

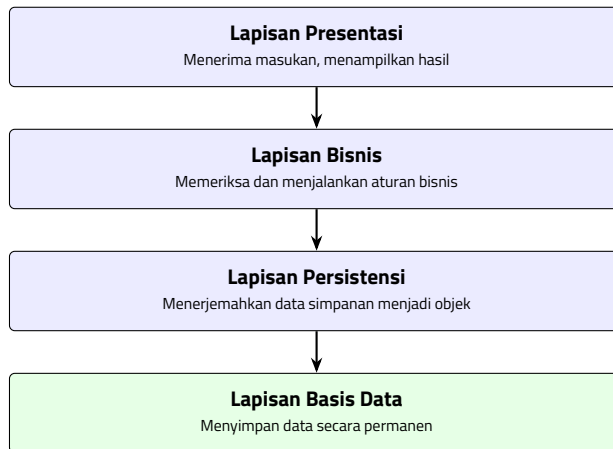
Salah satu alasan pemakaiannya bersifat organisasi, bukan teknis. Pembagian lapisan mengikuti pembagian kerja yang sudah ada di banyak tim pengembang. Ada yang mengerjakan antarmuka, ada yang mengerjakan aturan bisnis, dan ada yang mengerjakan basis data. Struktur tim tersebut memetakan diri dengan rapi ke struktur lapisan, sehingga setiap orang tahu berkas mana yang menjadi tanggung jawabnya.

Kemudahan itu ada harganya. Ketika aplikasi membesar, pemisahan yang tadinya menolong justru memperlambat perubahan. Satu fitur baru sering menyentuh

keempat lapisan sekaligus. Bab ini membahas pembagian lapisan, aturan ketergantungan di antaranya, serta pilihan antara lapisan tertutup dan lapisan terbuka. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [1].

2.2 Pembagian Lapisan

Bentuk baku *layered architecture* terdiri atas empat lapisan. Gambar 2.1 menunjukkan susunannya beserta arah panggilan yang diizinkan. Panah hanya menunjuk ke bawah, dan tidak ada panah yang melompati satu lapisan.



Gambar 2.1: Empat lapisan baku beserta arah panggilan yang diizinkan

Lapisan presentasi menangani segala hal yang dilihat pengguna. Lapisan ini menerima masukan, memanggil lapisan bisnis, lalu menampilkan hasilnya. Lapisan ini tidak mengetahui asal data maupun cara data disimpan.

Lapisan bisnis memuat aturan yang membuat sistem tersebut bernilai. Pemeriksaan kelayakan, perhitungan, dan keputusan proses berada di sini. Lapisan ini tidak mengetahui bentuk antarmuka maupun bentuk tabel basis data.

Lapisan persistensi menjembatani objek di dalam program dengan baris di dalam basis data. Kueri Structured Query Language (SQL) ditulis di lapisan ini, sehingga lapisan bisnis tidak perlu mengetahui nama tabel.

Lapisan basis data menyimpan data secara permanen. Lapisan ini biasanya berupa perangkat lunak tersendiri, bukan kode yang ditulis tim pengembang.

Pembagian tersebut memberi satu keuntungan yang jelas. Perubahan pada satu urusan teknis tertahan di satu lapisan. Penggantian basis data, misalnya, cukup menyentuh lapisan persistensi selama bentuk objek yang dikembalikannya tetap. Tabel 2.1 merangkum tanggung jawab dan larangan setiap lapisan.

Tabel 2.1: Tanggung jawab dan larangan setiap lapisan

Lapisan	Tanggung jawab	Tidak boleh mengetahui
Presentasi	Menerima masukan dan menampilkan hasil	Nama tabel, kueri, sumber data
Bisnis	Memeriksa aturan dan menghitung hasil	Bentuk antarmuka, bentuk tabel
Persistensi	Menerjemahkan baris menjadi objek	Aturan bisnis, kebutuhan tampilan
Basis Data	Menyimpan data secara permanen	Seluruh lapisan di atasnya

2.3 Aturan Ketergantungan

Aturan pokok *layered architecture* hanya satu. Sebuah lapisan boleh memanggil lapisan tepat di bawahnya, dan tidak boleh memanggil ke atas. Aturan itu terdengar sederhana, tetapi menentukan hampir seluruh sifat arsitektur ini.

Analogi yang mendekati adalah loket pembayaran uang kuliah di kampus. Mahasiswa berbicara kepada petugas loket, petugas loket berbicara kepada bagian keuangan, lalu bagian keuangan berbicara kepada bank. Mahasiswa tidak menelepon bank secara langsung, dan bank tidak memanggil mahasiswa. Setiap pihak hanya mengenal tetangga terdekatnya.

Aturan tersebut punya dua konsekuensi praktis. Pertama, satu lapisan dapat diganti tanpa menyentuh lapisan lain, selama bentuk panggilannya tetap sama. Kedua, satu permintaan yang sederhana tetap harus melewati seluruh lapisan, walaupun sebagian lapisan tidak mengerjakan apa pun.

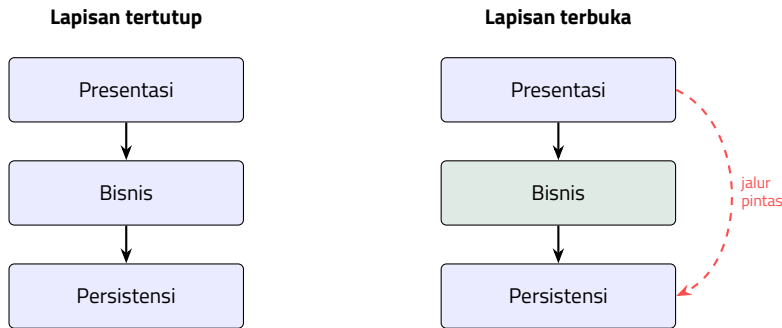
Konsekuensi kedua itulah yang menimbulkan persoalan berikutnya. Pertanyaannya, apakah setiap permintaan benar-benar wajib melewati semua lapisan.

2.4 Lapisan Tertutup dan Lapisan Terbuka

Dua jawaban atas pertanyaan tersebut menghasilkan dua bentuk yang berbeda. Gambar 2.2 menunjukkan keduanya berdampingan.

Pada **lapisan tertutup**, setiap panggilan wajib melewati seluruh lapisan tanpa perkecualian. Susunan ini menjaga arah ketergantungan tetap bersih. Harganya muncul pada permintaan yang tidak punya aturan bisnis sama sekali. Lapisan bisnis lalu hanya meneruskan panggilan tanpa menambahkan apa pun.

Keadaan tersebut dikenal sebagai *architecture sinkhole anti-pattern*, yaitu permintaan yang mengalir melewati beberapa lapisan tanpa satu pun lapisan mengerjakan sesuatu. Permintaan hanya turun untuk mengambil data, lalu naik kembali tanpa diolah. Richards dan Ford [1] menyebut pegangan delapan puluh berbanding dua puluh. Sekurang-kurangnya delapan puluh persen permintaan seharusnya



Gambar 2.2: Lapisan tertutup mewajibkan setiap panggilan melewati lapisan bisnis, sedangkan lapisan terbuka mengizinkan jalur pintas

benar-benar memakai lapisan bisnis, dan kurang dari dua puluh persen boleh sekadar diteruskan. Keadaan ini hampir tidak mungkin dihapus sepenuhnya, sehingga yang diusahakan adalah menekan jumlahnya, bukan menghilangkannya.

Satu hal lain sering terlupakan. Lapisan mana yang tertutup dan lapisan mana yang terbuka wajib dicatat dan disepakati satu tim. Tanpa catatan tersebut, setiap orang menebak sendiri aturan yang berlaku, dan sistem berakhir dengan ketergantungan yang rapat serta sulit diuji.

Pada **lapisan terbuka**, permintaan tanpa aturan bisnis boleh memangkas satu lompatan. Lapisan presentasi memanggil lapisan persistensi secara langsung. Bentuk ini memangkas kode penerus, tetapi menimbulkan ketergantungan baru. Lapisan presentasi kini mengenal lapisan persistensi, sehingga perubahan pada lapisan persistensi dapat merambat ke atas.

Listing 2.1 memperlihatkan fungsi penerus tersebut dalam bentuk kode. Fungsi itu ada semata-mata untuk mematuhi aturan lapisan tertutup.

```

1 def susun_daftar_kurs():
2     """Meneruskan panggilan tanpa menambahkan apa pun.
3
4     Fungsi seperti inilah yang disebut lapisan penerus. Fungsi ini ada hanya
5     untuk mematuhi aturan lapisan tertutup, bukan karena ada aturan bisnis.
6     """
7     return daftar_kurs()

```

Listing 2.1: Fungsi penerus pada lapisan bisnis, tersedia di `sources/chapter02/aplikasi_tertutup.py`

Bandingkan fungsi tersebut dengan Listing 2.2. Fungsi kedua ini benar-benar mengerjakan sesuatu, yaitu memeriksa dua aturan sebelum menghitung. Perbedaan keduanya menjadi dasar untuk memutuskan apakah sebuah panggilan layak dipintas.

```

1 def hitung_konversi(kode_asal, kode_tujuan, nominal):

```

```
2      """Menghitung hasil konversi beserta pemeriksaan aturan bisnisnya.
3
4      Aturan yang diperiksa ada dua, yaitu nominal harus positif dan tidak boleh
5      melewati batas wajar. Keduanya diperiksa di sini, bukan di lapisan
        antarmuka.
6      """
7      if nominal <= 0:
8          raise ValueError("Nominal harus lebih besar dari nol")
9      if nominal > BATAS_NOMINAL_WAJAR:
10         raise ValueError("Nominal melewati batas wajar")
11     nilai = cari_kurs(kode_asal, kode_tujuan)
12     if nilai is None:
13         raise LookupError(PESAN_KURS_HILANG.format(kode_asal, kode_tujuan))
14     return nominal * nilai
```

Listing 2.2: Fungsi bisnis yang benar-benar bekerja, tersedia di `sources/chapter02/aplikasi_tertutup.py`

2.5 Kelebihan dan Kekurangan

Tabel 2.2 merangkum untung rugi kedua bentuk tersebut. Perbandingan ini menjadi bahan pertimbangan sebelum memilih salah satunya.

Tabel 2.2: Perbandingan lapisan tertutup dan lapisan terbuka

Aspek	Lapisan tertutup	Lapisan terbuka
Arah ketergantungan	Bersih, hanya ke lapisan tepat di bawah	Presentasi ikut mengenal persis-tensi
Jumlah kode	Bertambah oleh fungsi penerus	Lebih sedikit
Kecepatan	Satu lompatan lebih banyak per permintaan	Satu lompatan lebih sedikit
Dampak perubahan	Tertahan di satu lapisan	Dapat merambat ke lapisan pre-sentasi
Cocok untuk	Sistem dengan aturan bisnis pa-dat	Pembacaan sederhana tanpa aturan bisnis

Kelebihan utama *layered architecture* secara umum ada tiga. Pembagian kerja menjadi jelas, pengujian setiap lapisan dapat berdiri sendiri, dan penggantian satu teknologi tidak menyentuh seluruh sistem. Ketiganya membuat bentuk ini cocok untuk sistem berukuran kecil sampai menengah.

Kekurangannya muncul pada sistem besar. Satu fitur baru sering menyentuh keempat lapisan sekaligus, sehingga tidak ada satu bagian pun yang dapat *deploy* sendiri. Selain itu seluruh lapisan tumbuh bersama-sama, sehingga sistem tetap berupa satu kesatuan yang harus dirilis sekaligus.

2.6 Ringkasan

Layered architecture membagi aplikasi menjadi empat lapisan, yaitu presentasi, bisnis, persistensi, dan basis data. Setiap lapisan mengerjakan satu jenis urusan teknis, lalu memanggil lapisan di bawahnya. Pembagian tersebut mengikuti pembagian kerja yang sudah ada di banyak tim pengembang, dan itulah salah satu alasan bentuk ini banyak dipakai.

Aturan pokoknya hanya satu, yaitu panggilan hanya boleh mengarah ke bawah. Aturan itu menjaga agar perubahan pada satu urusan teknis tertahan di satu lapisan. Penggantian basis data cukup menyentuh lapisan persistensi, selama bentuk objek yang dikembalikannya tidak berubah.

Aturan yang sama menimbulkan biaya pada permintaan sederhana. Permintaan yang tidak punya aturan bisnis tetap harus melewati lapisan bisnis, sehingga muncul fungsi yang hanya meneruskan panggilan. Perbandingan delapan puluh berbanding dua puluh dipakai sebagai pegangan. Bila fungsi penerus sudah lebih banyak daripada fungsi yang benar-benar bekerja, susunan lapisannya perlu ditinjau.

Lapisan terbuka menawarkan jalan keluar dengan mengizinkan jalur pintas untuk permintaan tanpa aturan bisnis. Kecepatan yang didapat biasanya kecil, dan hasil pengukuran pada latihan bab ini menunjukkan selisihnya dapat tenggelam oleh ragam pengukuran. Yang lebih terasa adalah ketergantungan baru antara lapisan presentasi dan lapisan persistensi.

Pilihan antara keduanya karena itu bukan soal kecepatan, melainkan soal siapa yang boleh mengenal siapa. Lapisan tertutup membeli kebersihan ketergantungan dengan kode penerus. Lapisan terbuka membeli kode yang lebih ringkas dengan ketergantungan yang lebih luas. Keputusan diambil dengan menghitung kedua hal tersebut, bukan dengan mengikuti kebiasaan.

2.7 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter02/`, tetapi menysasar tiga masalah yang berbeda. Latihan 1 memeriksa lapisan yang tidak bekerja. Latihan 2 memeriksa arah ketergantungan yang dilanggar. Latihan 3 mengukur biaya waktu satu lompatan antar lapisan. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya, bukan pada kecepatan pengerjaan. Hasil pengukuran setiap mahasiswa akan berbeda, dan yang dilaporkan adalah pola perbandingannya, bukan angka persisnya.

Seluruh latihan dijalankan seperti pada Listing 2.3.

Listing 2.3: Persiapan sebelum ketiga latihan

```
1 cd sources/chapter02
2 python basis_data.py
```

Latihan 1: Menghitung Fungsi Penerus

Latihan ini membuktikan berapa besar bagian lapisan bisnis yang hanya meneruskan panggilan. Pegangan delapan puluh berbanding dua puluh sebenarnya dihitung dari jumlah permintaan. Pada aplikasi sekecil ini setiap fungsi melayani tepat satu jalur permintaan, sehingga menghitung fungsi sudah cukup mewakili. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python periksa_lapisan.py aplikasi_tertutup.py`, lalu catat baris Fungsi penerus dan Bagian penerus.
- 2. Jalankan perintah yang sama untuk `aplikasi_terbuka.py`, lalu catat kedua baris tersebut.
- 3. Buka kedua berkas, lalu periksa tubuh setiap fungsi yang dilaporkan sebagai penerus.
- 4. Bandingkan bagian penerus pada kedua berkas dengan pegangan delapan puluh berbanding dua puluh.

Tabel 2.3 memperlihatkan hasil satu kali pemeriksaan nyata.

Tabel 2.3: Contoh hasil Latihan 1 yang sudah terisi

Berkas	Fungsi	Penerus	Bagian
aplikasi_tertutup.py	8	1	12,5 persen
aplikasi_terbuka.py	7	0	0,0 persen

Isikan hasil pemeriksaan pada Tabel 2.4.

Tabel 2.4: Lembar isian Latihan 1

Berkas	Fungsi	Penerus	Bagian
aplikasi_tertutup.py	...	...	...
aplikasi_terbuka.py	...	...	...
Nama fungsi penerus	...		

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, apakah bagian penerus pada versi tertutup sudah memenuhi pegangan delapan puluh berbanding dua puluh. Kedua, versi terbuka melaporkan nol fungsi penerus, sehingga jelaskan ke mana pekerjaan fungsi tersebut berpindah.

Latihan 2: Menemukan Pelanggaran Arah Ketergantungan

Latihan ini membuktikan bahwa jalur pintas menimbulkan ketergantungan baru. Skrip `periksa_lapisan.py` menghitung selisih indeks lapisan antara pemanggil dan yang dipanggil, lalu menandai selisih dua atau lebih sebagai pelanggaran. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python periksa_lapisan.py` untuk kedua berkas aplikasi, lalu catat baris Pelanggaran arah.
- 2. Salin nama fungsi pemanggil dan fungsi yang dipanggil pada setiap pelanggaran, beserta nama lapisannya.
- 3. Buka `sources/chapter02/aplikasi_terbuka.py`, lalu temukan baris yang menimbulkan pelanggaran tersebut.
- 4. Tambahkan satu fungsi baru pada lapisan presentasi yang membaca basis data langsung, lalu jalankan ulang pemeriksaannya.

Tabel 2.5 memperlihatkan hasil satu kali pemeriksaan nyata.

Tabel 2.5: Contoh hasil Latihan 2 yang sudah terisi

Berkas	Pelanggaran	Rincian
<code>aplikasi_tertutup.py</code>	0	tidak ada
<code>aplikasi_terbuka.py</code>	1	<code>tampilkan_daftar_kurs</code> (presentasi) memanggil <code>daftar_kurs</code> (persistensi)

Isikan hasil pemeriksaan pada Tabel 2.6.

Tabel 2.6: Lembar isian Latihan 2

Berkas	Pelanggaran	Rincian
<code>aplikasi_tertutup.py</code>	...	...
<code>aplikasi_terbuka.py</code>	...	...
Setelah fungsi baru ditambahkan	...	...

Jawab dua pertanyaan berikut dengan menunjuk isi tabel. Pertama, Latihan 1 menunjukkan versi terbuka lebih ringkas, sedangkan latihan ini menunjukkan versi terbuka melanggar arah ketergantungan, sehingga jelaskan mana yang lebih mahal bagi tim yang merawat kode tersebut. Kedua, jelaskan akibatnya bila lapisan persistensi diganti, dilihat dari daftar pelanggaran yang diperoleh.

Latihan 3: Mengukur Biaya Satu Lompatan

Latihan ini membuktikan berapa besar waktu yang dihemat oleh jalur pintas. Langkah kerjanya sebagai berikut.

1. Jalankan `python ukur_lapisan.py`, lalu catat median dan persentil ke-95 untuk kedua jalur.
2. Ulangi pengukuran sebanyak tiga kali, lalu catat ketiga hasilnya.
3. Hitung selisih antara pengukuran terbesar dan terkecil pada satu jalur yang sama.
4. Bandingkan selisih tersebut dengan selisih antar kedua jalur.

Tabel 2.7 memperlihatkan hasil satu kali pengukuran nyata. Angkanya disalin apa adanya, termasuk kejanggalannya.

Tabel 2.7: Contoh hasil Latihan 3 yang sudah terisi

Jalur	Median (ms)	Persentil ke-95 (ms)
Tertutup, lewat lapisan bisnis	0,0710	0,0970
Terbuka, jalur pintas	0,0757	0,0958
Selisih	-0,0047	+0,0012

Angka tersebut janggal. Jalur pintas seharusnya lebih cepat, tetapi mediannya justru lebih besar, sedangkan persentil ke-95 justru lebih kecil. Penyebabnya adalah besar selisih yang dicari. Satu pemanggilan fungsi Python berbiaya sekitar sepersepuluh mikrodetik, sedangkan satu pembacaan basis data pada pengukuran ini menghabiskan sekitar tujuh puluh mikrodetik. Selisih yang dicari karena itu lebih kecil daripada ragam antar pengukuran, sehingga urutan hasilnya dapat berbalik. Kejanggalan ini justru merupakan temuan utama latihan tersebut.

Isikan hasil pengukuran pada Tabel 2.8.

Tabel 2.8: Lembar isian Latihan 3

Jalur dan pengukuran ke-	Median (ms)	Persentil ke-95 (ms)
Tertutup, ke-1	...	...
Terbuka, ke-1	...	...
Tertutup, ke-2	...	...
Terbuka, ke-2	...	...
Tertutup, ke-3	...	...
Terbuka, ke-3	...	...
Ragam terbesar pada satu jalur	...	...

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, apakah jalur pintas selalu lebih cepat pada ketiga pengukuran. Kedua, apakah ragam antar pengukuran pada satu jalur lebih besar daripada selisih antar kedua jalur. Ketiga, dengan menggabungkan hasil ketiga latihan, tentukan bentuk mana yang dipilih untuk aplikasi tersebut, lalu sebutkan angka mana yang menjadi dasar keputusannya.

Bab 3

Model-View-*

Tujuan Pembelajaran

1. Menjelaskan pembagian peran pada keempat varian Model-View-*, yaitu MVC, MVP, MVVM, dan MVI, beserta arah ketergantungan di antara komponennya.
2. Membedakan keempat varian tersebut berdasarkan komponen yang dikenal oleh View, lalu menentukan varian mana yang sesuai untuk sebuah kebutuhan.
3. Mengukur jumlah ketergantungan, kemudahan pengujian tanpa antarmuka, dan luas dampak perubahan pada keempat varian.

3.1 Pendahuluan

Antarmuka pengguna berubah jauh lebih sering daripada aturan bisnis. Warna, susunan tombol, dan urutan layar dapat berganti beberapa kali dalam satu semester, sedangkan aturan perhitungannya tetap. Mencampur keduanya di dalam satu berkas membuat setiap perubahan tampilan berisiko merusak perhitungan.

Kelompok pola Model-View-* lahir untuk memisahkan keduanya. Seluruh varian membagi aplikasi menjadi bagian yang menyimpan data, bagian yang menampilkan, dan bagian yang menghubungkan keduanya. Yang membedakan antar varian hanyalah bagaimana bagian penghubung tersebut bekerja, dan komponen mana yang boleh dikenal oleh View.

Bab ini membahas empat varian yang paling banyak dipakai, yaitu Model-View-Controller (MVC), Model-View-Presenter (MVP), Model-View-ViewModel (MVVM), dan Model-View-Intent (MVI). Keempatnya diperbandingkan memakai satu aplika-

si yang sama, sehingga perbedaannya dapat diukur, bukan sekadar diperdebatkan. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [1].

3.2 Peran Ketiga Komponen

Seluruh varian Model-View-* berangkat dari pembagian yang sama. Tabel 3.1 merangkum ketiga peran dasarnya.

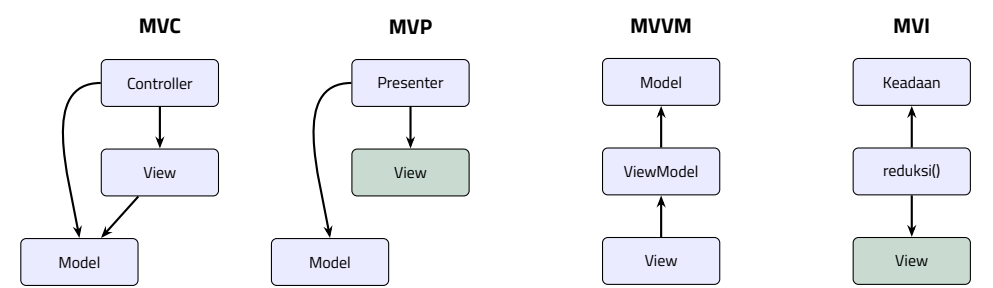
Tabel 3.1: Peran dasar yang dipakai seluruh varian Model-View-*

Peran	Tanggung jawab
Model	Menyimpan data dan menjalankan aturan bisnis
View	Menampilkan data dan menerima tindakan pengguna
Penghubung	Menerjemahkan tindakan pengguna menjadi perubahan pada Model

Nama peran penghubung berbeda pada setiap varian. Pada MVC namanya Controller, pada MVP namanya Presenter, pada MVVM namanya ViewModel, sedangkan pada MVI peran tersebut dipegang oleh sebuah fungsi reduksi, bukan oleh sebuah kelas. Perbedaan nama tersebut tidak penting. Yang menentukan adalah arah ketergantungan di antara ketiganya.

3.3 Empat Varian dan Arah Ketergantungannya

Gambar 3.1 menunjukkan keempat varian berdampingan. Panah menyatakan arah ketergantungan, yaitu komponen yang menunjuk mengenal komponen yang ditunjuk.



Gambar 3.1: Arah ketergantungan pada keempat varian. Kotak hijau menandai View yang tidak bergantung pada komponen mana pun

MVC menempatkan Controller sebagai penerima masukan. Controller memanggil Model, lalu View membaca Model untuk mengambil nilai yang akan ditampilkan. Ciri khasnya, View mengenal Model.

MVP menjadikan View pasif. Presenter mengerjakan seluruh penyusunan teks, lalu mendorong hasilnya ke View. View tidak mengenal Model sama sekali, dan hanya menyimpan apa yang diberikan Presenter.

MVVM membalik arah pemberitahuan. ViewModel menyediakan properti yang dapat diamati, lalu View mendaftarkan diri sebagai pengamat. Ketika properti berubah, View memperbarui dirinya sendiri. ViewModel tidak mengenal View.

MVI menghapus kelas penghubung sama sekali. Setiap tindakan pengguna dibungkus menjadi Intent, lalu sebuah fungsi reduksi mengubah pasangan keadaan lama dan Intent menjadi keadaan baru yang tidak dapat diubah lagi. Listing 3.1 memperlihatkan fungsi tersebut.

```

1 def reduksi(keadaan, intent):
2     """Mengubah keadaan lama dan sebuah Intent menjadi keadaan baru.
3
4     Fungsi ini murni, artinya keluarannya hanya bergantung pada kedua
5     masukannya.
6     Sifat itulah yang membuat riwayat tampilan dapat diputar ulang.
7     """
8     try:
9         hasil = domain.hitung_konversi(intent.kode_asal, intent.kode_tujuan,
10                                       intent.nominal)
11         return replace(keadaan, teks=f"Hasil: {hasil:,.2f}", sedang_sibuk=False)
12     except (ValueError, LookupError) as kesalahan:
13         return replace(keadaan, teks=f"Galat: {kesalahan}", sedang_sibuk=False)

```

Listing 3.1: Fungsi reduksi pada MVI, tersedia di `sources/chapter03/mvi.py`

Karena fungsi tersebut murni, seluruh keadaan yang pernah digambar dapat disimpan lalu diputar ulang persis seperti semula. Sifat itu membuat antarmuka yang keluarannya berubah-ubah tetap dapat diuji secara pasti.

3.4 Ketergantungan View sebagai Pembeda

Satu pertanyaan cukup untuk membedakan keempat varian, yaitu komponen apa yang dikenal oleh View. Jawabannya menentukan seberapa mudah logika tampilan diuji tanpa menjalankan antarmuka sungguhan.

Analogi yang mendekati adalah loket informasi di kampus. Petugas loket yang harus membuka lemari arsip sendiri sulit digantikan, sebab penggantinya harus mengenal isi lemari tersebut. Petugas yang hanya membacakan lembar yang sudah disiapkan bagian lain mudah digantikan siapa pun. View pada MVP dan MVI berperan seperti petugas kedua.

Tabel 3.2 merangkum hasil pemeriksaan nyata memakai `periksa_mv.py`. Kolom terakhir menunjukkan cara pengujian yang paling sedikit menyentuh antarmuka.

Tabel 3.2: Ketergantungan dan cara pengujian keempat varian

Varian	Total	View butuh	Cara pengujian
MVC	3	Model	Membutuhkan View asli
MVP	2	tidak ada	Cukup View tiruan
MVVM	3	ViewModel	Tanpa View, cukup membaca propertinya
MVI	0	tidak ada	Tanpa View, cukup memanggil fungsi reduksi

Angka pada tabel tersebut punya dua bacaan. Pertama, MVI mencatat nol ketergantungan karena penghubungnya berupa fungsi, bukan kelas yang menyimpan rujukan ke komponen lain. Kedua, MVVM mencatat total yang sama dengan MVC, tetapi ketergantungannya menunjuk ke arah yang berbeda. Pada MVC, View mengenal Model. Pada MVVM, View mengenal ViewModel, sedangkan ViewModel tidak mengenal View. Jumlah yang sama karena itu tidak berarti sifatnya sama.

3.5 Kelebihan dan Kekurangan

Tabel 3.3 merangkum untung rugi keempat varian tersebut.

Tabel 3.3: Perbandingan keempat varian Model-View-*

Varian	Kelebihan	Kekurangan
MVC	Paling sederhana, banyak didukung framework	View mengenal Model, sehingga sulit diuji terpisah
MVP	View pasif dan mudah diganti tiruan	Presenter mudah membengkok karena memuat seluruh penyusunan teks
MVVM	Pembaruan tampilan berjalan otomatis lewat pengamatan	Membutuhkan mekanisme properti yang dapat diamati
MVI	Keadaan tetap, riwayat dapat diputar ulang	Setiap tindakan harus dibungkus menjadi Intent, sehingga kode bertambah

Pemilihan varian jarang bebas sepenuhnya. Framework antarmuka biasanya sudah menganjurkan satu varian tertentu, dan melawan anjuran tersebut berbiaya mahal. Yang tetap dapat dipilih adalah seberapa banyak logika yang dititipkan ke View. Semakin sedikit logika di dalam View, semakin besar bagian aplikasi yang dapat diuji tanpa menjalankan antarmuka.

3.6 Ringkasan

Kelompok pola Model-View-* memisahkan tampilan dari aturan bisnis. Seluruh varian membagi aplikasi menjadi bagian yang menyimpan data, bagian yang menampilkan, dan bagian yang menghubungkan keduanya. Perbedaan antar varian terletak pada cara bagian penghubung bekerja, bukan pada tujuannya.

Pembeda yang paling menentukan adalah komponen yang dikenal oleh View. Pada MVC View mengenal Model, pada MVVM View mengenal ViewModel, sedangkan pada MVP dan MVI View tidak mengenal apa pun. Semakin sedikit yang dikenal View, semakin besar bagian aplikasi yang dapat diuji tanpa menjalankan antarmuka.

Pengukuran pada bab ini menunjukkan jumlah ketergantungan saja tidak cukup untuk menilai sebuah varian. MVC dan MVVM sama-sama mencatat tiga ketergantungan, tetapi arahnya berbeda, dan perbedaan arah itulah yang menentukan kemudahan pengujiannya. Angka perlu dibaca bersama arahnya.

MVI mencatat nol ketergantungan karena penghubungnya berupa fungsi murni. Sifat murni tersebut memberi keuntungan tambahan, yaitu seluruh keadaan yang pernah digambar dapat disimpan lalu diputar ulang. Harganya, setiap tindakan pengguna harus dibungkus menjadi Intent, sehingga jumlah kode bertambah.

Pemilihan varian pada praktiknya lebih sering ditentukan oleh framework yang dipakai daripada oleh pertimbangan bebas. Yang tetap dapat dikendalikan adalah banyaknya logika yang dititipkan ke View. Keputusan itu diambil dengan melihat angka ketergantungan beserta arahnya, bukan dengan mengikuti kebiasaan.

3.7 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter03/`, tetapi menasar tiga masalah yang berbeda. Latihan 1 memeriksa siapa mengenal siapa. Latihan 2 memeriksa kemudahan pengujian tanpa antarmuka. Latihan 3 mengukur luas dampak satu permintaan perubahan. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Latihan 1: Memetakan Ketergantungan Antar Komponen

Latihan ini membuktikan bahwa keempat varian berbeda pada arah ketergantungannya, bukan hanya pada nama komponennya. Langkah kerjanya sebagai berikut.

1. Jalankan skrip pemeriksaan seperti pada Listing 3.2.
2. Catat baris `Total` ketergantungan dan `Ketergantungan View` untuk keempat varian.
3. Buka keempat berkas varian, lalu tunjukkan baris konstruktor yang menjadi sumber setiap ketergantungan tersebut.

4. Gambarkan arah panah antar komponen untuk keempat varian.

Listing 3.2: Menjalankan pemeriksaan ketergantungan

```
1 cd sources/chapter03
2 python periksa_mv.py
```

Tabel 3.4 memperlihatkan hasil satu kali pemeriksaan nyata.

Tabel 3.4: Contoh hasil Latihan 1 yang sudah terisi

Varian	Total	View butuh	Komponen lain
MVC	3	1	Controller butuh Model dan View
MVI	0	0	Penghubungnya berupa fungsi

Isikan hasil pemeriksaan pada Tabel 3.5.

Tabel 3.5: Lembar isian Latihan 1

Varian	Total	View butuh	Komponen lain
MVC	...	...	...
MVP	...	...	...
MVVM	...	...	...
MVI	...	...	...

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, MVC dan MVVM mencatat total yang sama, sehingga jelaskan mengapa keduanya tetap berbeda sifatnya. Kedua, jelaskan penyebab MVI mencatat nol ketergantungan.

Latihan 2: Menguji Tanpa Antarmuka

Latihan ini membuktikan hubungan antara ketergantungan View dan kemudahan pengujian. Langkah kerjanya sebagai berikut.

- 1. Jalankan `python uji_tanpa_antarmuka.py`, lalu catat cara uji yang dipakai setiap varian.
- 2. Buka `sources/chapter03/uji_tanpa_antarmuka.py`, lalu bandingkan isi keempat fungsi ujinya.
- 3. Coba tulis ulang pengujian MVC tanpa membuat objek View, lalu catat apa yang menghalangi.
- 4. Hitung berapa baris kode tambahan yang dibutuhkan setiap varian untuk dapat diuji.

Tabel 3.6 memperlihatkan hasil satu kali pengujian nyata.

Isikan hasil pengujian pada Tabel 3.7.

Tabel 3.6: Contoh hasil Latihan 2 yang sudah terisi

Varian	Lulus	Cara uji
MVC	ya	Membutuhkan View asli
MVI	ya	Tanpa View, cukup memanggil fungsi reduksi

Tabel 3.7: Lembar isian Latihan 2

Varian	Lulus	Cara uji dan baris tambahan
MVC	...	...
MVP	...	...
MVVM	...	...
MVI	...	...

Jawab dua pertanyaan berikut. Pertama, cocokkan kolom cara uji pada latihan ini dengan kolom ketergantungan View pada Latihan 1, lalu jelaskan hubungan antara keduanya. Kedua, sebutkan varian yang paling murah diuji beserta alasannya menurut angka yang diperoleh.

Latihan 3: Mengukur Luas Dampak Perubahan

Latihan ini membuktikan berapa banyak berkas yang tersentuh oleh satu permintaan perubahan yang sama. Permintaannya adalah menambahkan baris kedua pada tampilan yang menyebutkan nilai kurs yang dipakai. Langkah kerjanya sebagai berikut.

- 1. Catat lebih dahulu berkas apa saja yang menurut perkiraan akan berubah pada setiap varian.
- 2. Terapkan perubahan tersebut pada keempat varian, satu per satu.
- 3. Jalankan kembali `python uji_tanpa_antarmuka.py` untuk memastikan keempatnya masih berjalan.
- 4. Hitung berkas, kelas, dan baris yang berubah pada setiap varian memakai perintah pada Listing 3.3.

Listing 3.3: Menghitung baris yang berubah

```
1 cd sources/chapter03
2 git diff --stat mvc.py mvp.py mvvm.py mvi.py
```

Tabel 3.8 memperlihatkan bentuk pengisian yang diharapkan, memakai angka perkiraan sebagai contoh.

Isikan hasil pengukuran pada Tabel 3.9.

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, varian mana yang paling sedikit tersentuh, dan apakah hasilnya sesuai perkiraan

Tabel 3.8: Contoh bentuk pengisian Latihan 3

Varian	Kelas	Baris	Komponen yang berubah
MVC	2	6	Model dan View
MVI	2	5	Keadaan dan fungsi reduksi

Tabel 3.9: Lembar isian Latihan 3

Varian	Kelas	Baris	Komponen yang berubah
MVC	...	...	...
MVP	...	...	...
MVVM	...	...	...
MVI	...	...	...
Perkiraan awal yang meleset	...		

awal. Kedua, jelaskan mengapa varian dengan ketergantungan paling sedikit belum tentu yang paling sedikit berubah. Ketiga, dengan menggabungkan hasil ketiga latihan, tentukan varian yang dipilih untuk aplikasi tersebut beserta angka yang menjadi dasar keputusannya.

Bab 4

Ports and Adapters

Tujuan Pembelajaran

1. Menjelaskan peran port dan adapter serta arah ketergantungan yang membuat logika inti bebas dari teknologi penyimpanan.
2. Membedakan port masukan dari port keluaran, lalu menentukan komponen mana yang menjadi adapter pada sebuah sistem.
3. Mengukur kemandirian logika inti, biaya penggantian adapter, dan cakupan pengujian tanpa infrastruktur.

4.1 Pendahuluan

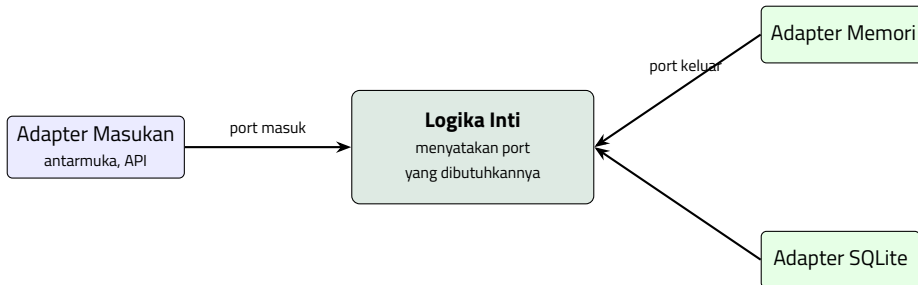
Bab 2 menunjukkan bahwa *layered architecture* menjaga arah panggilan tetap ke bawah. Aturan tersebut menahan perubahan di satu lapisan, tetapi tidak menghilangkan satu persoalan. Lapisan bisnis tetap memanggil lapisan persistensi, sehingga lapisan bisnis tetap mengenal keberadaan penyimpanan.

Ports and adapters, yang juga disebut *hexagonal architecture*, menyelesaikan persoalan tersebut dengan membalik arah ketergantungan. Logika inti tidak lagi memanggil penyimpanan. Logika inti justru menyatakan apa yang dibutuhkannya dalam bentuk antarmuka, lalu pihak lain yang menyediakannya. Antarmuka itu disebut port, dan pengisinya disebut adapter.

Pembalikan tersebut menghasilkan satu klaim yang tidak biasa, sebab klaim itu menyebut angka yang pasti. Jumlah teknologi penyimpanan yang disebut oleh logika inti harus nol. Klaim yang menyebut angka dapat dibuktikan salah, dan bab ini membuktikannya lewat pengukuran. Pembahasan mengikuti kerangka yang disusun oleh Richards dan Ford [1].

4.2 Port dan Adapter

Gambar 4.1 menunjukkan susunan dasarnya. Logika inti berada di tengah. Seluruh panah ketergantungan menunjuk ke dalam, sehingga tidak ada panah yang keluar dari logika inti.



Gambar 4.1: Seluruh panah menunjuk ke dalam, sehingga logika inti tidak pernah mengenal adapter mana pun

Port adalah antarmuka yang ditulis oleh logika inti untuk menyatakan kebutuhannya. Port tidak menyebut teknologi apa pun. Port hanya menyebut kelakuan yang dibutuhkan, misalnya kemampuan mencari nilai kurs.

Adapter adalah pengisi port tersebut untuk satu teknologi tertentu. Satu port dapat diisi oleh banyak adapter sekaligus, misalnya satu untuk penyimpanan dalam memori dan satu lagi untuk basis data.

Port dibedakan menjadi dua arah. **Port masukan** dipakai pihak luar untuk memanggil logika inti, misalnya antarmuka pengguna atau API. **Port keluaran** dipakai logika inti untuk meminta sesuatu dari luar, misalnya membaca data. Bab ini menitikberatkan port keluaran, sebab di situlah ketergantungan pada teknologi paling sering bocor.

Listing 4.1 memperlihatkan port keluaran pada contoh bab ini. Port tersebut hanya menyebut satu metode, dan tidak menyebut basis data sama sekali.

```

1 class SumberKurs(Protocol):
2     """Port keluaran yang dibutuhkan logika inti.
3
4     Siapa pun yang menyediakan metode ini dapat dipasang, baik penyimpanan
5     dalam
6     memori, basis data, maupun tiruan untuk pengujian.
7     """
8     def nilai(self, kode_asal, kode_tujuan):
9         """Mengembalikan nilai kurs, atau None bila pasangannya tidak ada."""

```

Listing 4.1: Port keluaran yang ditulis oleh logika inti, tersedia di `sources/chapter04/domain.py`

4.3 Menerima Adapter sebagai Argumen

Port saja belum cukup. Logika inti juga tidak boleh membuatnya sendiri. Bila logika inti memanggil konstruktor sebuah adapter, logika inti kembali mengenal teknologinya, dan seluruh manfaatnya hilang.

Analogi yang mendekati adalah stopkontak listrik di dinding. Stopkontak menentukan bentuk colokan yang diterimanya, tetapi tidak menentukan alat apa yang dicolokkan. Kipas angin, lampu, dan pengisi daya telepon sama-sama dapat dipasang. Dinding tidak perlu diubah setiap kali alatnya berganti.

Karena itu adapter diserahkan dari luar sebagai argumen. Listing 4.2 memperlihatkan caranya. Parameter pertama adalah sumber kurs, dan fungsi tersebut tidak pernah tahu adapter mana yang sedang dipakainya.

```

1 def hitung_konversi(sumber, kode_asal, kode_tujuan, nominal):
2     """Menghitung hasil konversi setelah memeriksa dua aturan bisnis.
3
4     Sumber kurs diterima sebagai argumen, bukan dibuat di dalam fungsi. Karena
5     itu
6     fungsi ini tidak pernah mengetahui teknologi penyimpanan yang dipakai.
7     """
8     if nominal <= 0:
9         raise ValueError("Nominal harus lebih besar dari nol")
10    if nominal > BATAS_NOMINAL_WAJAR:
11        raise ValueError("Nominal melewati batas wajar")
12    kurs = sumber.nilai(kode_asal, kode_tujuan)
13    if kurs is None:
14        raise LookupError(f"Kurs {kode_asal} ke {kode_tujuan} tidak tersedia")
15    return nominal * kurs

```

Listing 4.2: Adapter diterima sebagai argumen, tersedia di `sources/chapter04/domain.py`

Penentuan adapter dipindahkan ke satu berkas perangkat. Pada contoh bab ini berkas tersebut adalah `sources/chapter04/aplikasi.py`. Hanya berkas itulah yang mengetahui kedua adapter sekaligus.

4.4 Tiga Akibat yang Dapat Diukur

Susunan tersebut membawa tiga akibat, dan ketiganya menghasilkan angka. Tabel 4.1 merangkumnya beserta hasil pengukuran nyata pada contoh bab ini.

Akibat pertama adalah yang paling penting, sebab angkanya dapat membuktikan klaim tersebut salah. Bila `domain.py` ternyata mengimpor `sqlite3`, klaim kemandirian gugur pada penerapan tersebut, seberapa pun rapi susunan berkasnya. Listing 4.3 memperlihatkan pemeriksaannya.

```

1 def modul_diimpor(nama_berkas):
2     """Mengumpulkan seluruh nama modul yang diimpor sebuah berkas."""
3     with open(nama_berkas, encoding="utf-8") as berkas:

```

Tabel 4.1: Tiga akibat susunan port dan adapter beserta hasil pengukurannya

Akibat	Angka	Cara mengukur
Logika inti tidak mengenal teknologi	0 adapter	Menghitung modul yang diimpor domain.py
Penggantian adapter tidak menyentuh logika inti	1 baris	Mengganti pilihan adapter pada berkas perangkat
Pengujian berjalan tanpa infrastruktur	0 basis data	Memasang adapter tiruan berisi satu metode

```
4     pohon = ast.parse(berkas.read(), filename=nama_berkas)
5     nama = []
6     for simpul in ast.walk(pohon):
7         if isinstance(simpul, ast.Import):
8             nama.extend(a.name for a in simpul.names)
9         elif isinstance(simpul, ast.ImportFrom) and simpul.module:
10            nama.append(simpul.module)
11     return nama
```

Listing 4.3: Pemeriksaan kemandirian logika inti, tersedia di `sources/chapter04/periksa_port.py`

4.5 Kelebihan dan Kekurangan

Tabel 4.2 merangkum untung rugi susunan ini dibandingkan *layered architecture* pada Bab 2.

Tabel 4.2: Perbandingan ports and adapters dengan layered architecture

Aspek	Layered	Ports and adapters
Arah ketergantungan	Ke bawah, menuju penyimpanan	Ke dalam, menuju logika inti
Pengetahuan logika inti	Mengenal lapisan persistensi	Tidak mengenal teknologi apapun
Penggantian penyimpanan	Menyentuh lapisan persistensi	Menambah satu adapter baru
Pengujian	Membutuhkan basis data atau tiruannya	Cukup adapter tiruan berisi satu metode
Jumlah berkas	Lebih sedikit	Bertambah oleh port dan adapter

Kekurangannya terletak pada jumlah berkas. Setiap kebutuhan keluar menambah satu port beserta sekurang-kurangnya satu adapter. Untuk aplikasi kecil dengan satu teknologi penyimpanan yang tidak akan berganti, tambahan tersebut belum tentu sepadan.

4.6 Ringkasan

Ports and adapters membalik arah ketergantungan yang berlaku pada *layered architecture*. Logika inti tidak lagi memanggil penyimpanan. Logika inti menyatakan kebutuhannya sebagai port, lalu adapter mengisinya. Seluruh panah ketergantungan karena itu menunjuk ke dalam.

Port hanya menyebut kelakuan yang dibutuhkan, bukan teknologi yang menyediakannya. Satu port dapat diisi banyak adapter sekaligus. Pada contoh bab ini, adapter memori dan adapter SQLite mengisi port yang sama, dan keduanya menghasilkan jawaban yang sama tanpa mengubah satu baris pun pada logika inti.

Susunan tersebut hanya bekerja bila logika inti tidak membuat adapternya sendiri. Adapter diserahkan dari luar sebagai argumen, dan pemilihannya dipusatkan pada satu berkas perangkat. Berkas perangkat itulah satu-satunya tempat yang mengetahui seluruh adapter.

Klaim utama pola ini menyebut angka yang pasti, yaitu nol teknologi penyimpanan yang dikenal logika inti. Klaim yang menyebut angka dapat dibuktikan salah oleh pemeriksaan sederhana. Pemeriksaan pada bab ini menunjukkan `domain.py` hanya mengimpor satu modul standar, sehingga klaimnya terpenuhi.

Keuntungan tersebut ada harganya, yaitu jumlah berkas yang bertambah. Setiap kebutuhan keluar menambah satu port beserta adapternya. Keputusan memakai pola ini diambil dengan membandingkan tambahan berkas tersebut terhadap kemungkinan pergantian teknologi di kemudian hari.

4.7 Latihan

Ketiga latihan berikut memakai satu basis kode yang sama di `sources/chapter04/`, tetapi menasar tiga masalah yang berbeda. Latihan 1 memeriksa kemandirian logika inti. Latihan 2 mengukur biaya penggantian adapter. Latihan 3 memeriksa pengujian tanpa infrastruktur. Penilaian didasarkan pada kelengkapan tabel hasil dan ketepatan jawaban analisisnya. Hasil setiap mahasiswa dapat berbeda, dan yang dilaporkan adalah pola perbandingannya.

Latihan 1: Membuktikan Kemandirian Logika Inti

Latihan ini menguji klaim utama pola tersebut, yaitu logika inti tidak mengenal teknologi penyimpanan apa pun. Langkah kerjanya sebagai berikut.

1. Jalankan pemeriksaan seperti pada Listing 4.4, lalu catat baris `Adapter` disebut.

- 2. Buka `sources/chapter04/domain.py`, lalu pastikan tidak ada nama teknologi penyimpanan di dalamnya.
- 3. Tambahkan baris `import sqlite3` pada `domain.py`, lalu jalankan ulang pemeriksaannya dan catat hasilnya.
- 4. Batalkan penambahan tersebut, lalu pastikan pemeriksaan kembali bersih.

Listing 4.4: Menjalankan pemeriksaan kemandirian

```
1 cd sources/chapter04
2 python periksa_port.py
```

Tabel 4.3 memperlihatkan hasil satu kali pemeriksaan nyata.

Tabel 4.3: Contoh hasil Latihan 1 yang sudah terisi

Keadaan berkas	Adapter disebut	Klaim
Apa adanya	0	terpenuhi
Setelah <code>import sqlite3</code> ditambahkan	1	gagal

Isikan hasil pemeriksaan pada Tabel 4.4.

Tabel 4.4: Lembar isian Latihan 1

Keadaan berkas	Adapter disebut	Klaim
Apa adanya	...	...
Setelah impor ditambahkan	...	...
Modul yang tetap boleh diimpor	...	

Jawab dua pertanyaan berikut. Pertama, `domain.py` tetap mengimpor satu modul standar, sehingga jelaskan mengapa impor tersebut tidak melanggar klaim. Kedua, jelaskan mengapa klaim yang menyebut angka lebih berguna daripada klaim yang hanya menyebut sifat.

Latihan 2: Mengukur Biaya Penggantian Adapter

Latihan ini membuktikan berapa banyak kode yang berubah ketika teknologi penyimpanan diganti. Langkah kerjanya sebagai berikut.

- 1. Jalankan aplikasi dengan kedua adapter seperti pada Listing 4.5, lalu pastikan hasilnya sama.
- 2. Catat berkas mana saja yang harus dibuka untuk berpindah adapter.
- 3. Tulis satu adapter ketiga yang membaca kurs dari berkas teks biasa.
- 4. Catat jumlah berkas dan baris yang berubah agar adapter ketiga tersebut dapat dipakai.

Listing 4.5: Menjalankan aplikasi dengan dua adapter

```
1 cd sources/chapter04
2 python aplikasi.py memori
3 python aplikasi.py sqlite
```

Tabel 4.5 memperlihatkan hasil satu kali pengukuran nyata.

Tabel 4.5: Contoh hasil Latihan 2 yang sudah terisi

Kegiatan	Berkas berubah	Baris
Berpindah dari memori ke SQLite	0	0
Menambah adapter ketiga	2	22

Isikan hasil pengukuran pada Tabel 4.6.

Tabel 4.6: Lembar isian Latihan 2

Kegiatan	Berkas berubah	Baris
Berpindah antar adapter yang sudah ada	...	...
Menambah adapter ketiga	...	...
Berkas domain.py ikut berubah	...	

Jawab dua pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, apakah domain.py ikut berubah ketika adapter ketiga ditambahkan. Kedua, bandingkan angka tersebut dengan biaya penggantian basis data pada Bab 2, lalu jelaskan penyebab perbedaannya.

Latihan 3: Menguji Tanpa Infrastruktur

Latihan ini membuktikan bahwa logika inti dapat diuji tanpa basis data. Langkah kerjanya sebagai berikut.

- 1. Jalankan python uji_tanpa_basis_data.py, lalu catat hasil ketiga pengujiannya.
- 2. Buka sources/chapter04/uji_tanpa_basis_data.py, lalu hitung jumlah baris kelas SumberTiruan.
- 3. Tambahkan satu pengujian baru untuk nominal yang melewati batas wajar.
- 4. Hitung berapa banyak infrastruktur yang disentuh seluruh pengujian tersebut.

Tabel 4.7 memperlihatkan hasil satu kali pengujian nyata.

Isikan hasil pengujian pada Tabel 4.8.

Jawab tiga pertanyaan berikut dengan menunjuk angka pada tabel. Pertama, berapa baris kode yang dibutuhkan untuk menggantikan seluruh basis data pada

Tabel 4.7: Contoh hasil Latihan 3 yang sudah terisi

Pengujian	Lulus	Basis data disentuh
Hasil benar	ya	0
Nominal nol ditolak	ya	0
Kurs hilang ditolak	ya	0

Tabel 4.8: Lembar isian Latihan 3

Pengujian	Lulus	Basis data disentuh
Hasil benar	...	...
Nominal nol ditolak	...	...
Kurs hilang ditolak	...	...
Nominal melewati batas	...	...
Jumlah baris adapter tiruan	...	

pengujian. Kedua, jelaskan mengapa jumlah baris tersebut menjadi sedikit. Ketiga, dengan menggabungkan hasil ketiga latihan, tentukan apakah tambahan port dan adapter sepadan untuk aplikasi sekecil ini, lalu sebutkan angka yang menjadi dasar keputusannya.

Bibliografi

- [1] Mark Richards and Neal Ford. *Fundamentals of software architecture: an engineering approach*. O'Reilly Media, 2020.